

DataCollection2-S26-SE

February 23, 2026

1 Team Name: B Team

2 Authors: Madelyne Dusbabek, Gregory Lowman, Jay Hall

3 Cell Number: R22

4 J-V Apparatus Number: JV3

5 EQE Apparatus Number: EQE1

5.1 J-V Analysis

1. Create a SINGLE plot of the J-V curves of the **forward scans** with data from all pixels.
 - a. Remember you can copy and paste code from previous notebooks and use AI tools.
 - b. Also remember that you measured I-V from the pixels, so you will need to determine the current density, J, first before plotting.

Question 1: Discuss what you observe from looking at the comparison of your J-V curves. Do you have any pixels showing different behaviors?

All of the cells seem to be reaching the short circuit current at slightly lower voltages this week. In both JV curves, pixel 6 is our leading cell. Pixel 8 and 4 followed closely last week, but have dropped a few mA/cm² below cell 6 this week. The maximum current also appears to be a little higher this week for most of the cells.

6 JV curve of forward scan after 2 weeks (and last weeks to compare)

```
[ ]: #Mount to google drive to use shared files

from google.colab import drive
drive.mount("/content/drive", force_remount=True)

#Import libraries to do math and build the plots necessary

import pandas as pd
import numpy as np
```

```

from matplotlib import pyplot as plt
import os

#Constant for the area of a solar cell

AREA = 0.14 # cm2

#Select a folder to pull CSV files from

folder = "/content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team B/Week4/
↳JV_Data"

#Create an array of files

files = sorted(os.listdir(folder))
jv_data = {}

#Loop through each file in the folder / array

for f in files:
    #Create appropriate file pathways

    df = pd.read_csv(folder + "/" + f)

    jv_columns = {
        "Voltage (V)": df["Voltage (V)"],
        "Current Density (mA/cm2)": df["Forward_mean (mA)"] / AREA
    }

    #Name each data set without the .csv
    name = f.split("/")[1].replace(".csv", "")
    jv_data[name] = pd.DataFrame(jv_columns)

#Create a plot for the data

plt.figure(figsize=(7,5))

#Plot all of the datasets (loop), not just one

for name, jv in jv_data.items():
    plt.plot(jv["Voltage (V)"], jv["Current Density (mA/cm2)"], ".",
↳label=name)

#Plot attributes

plt.axhline(0, linewidth=1)
plt.axvline(0, linewidth=1)

```

```

plt.xlabel("Voltage (V)")
plt.ylabel("Current Density (mA/cm$^2$)")
plt.title("Forward Scan J-V Curves (All Pixels)")
plt.legend(fontsize=7)
plt.grid(True)

# I inverted the axis bc that's how they showed this data in lecture
plt.gca().invert_yaxis()
plt.show()

##### TO COMPARE, I ADDED THE CODE FROM THE PREVIOUS WEEK_
↳#####
#####

folder = "/content/drive/MyDrive/Section 104 Tuesday 2:15 PM Team B/Week3/
↳JV_Data"

#Create an array of files

files = sorted(os.listdir(folder))
jv_data = {}

#Loop through each file in the folder / array

for f in files:
    #Create appropriate file pathways

    df = pd.read_csv(folder + "/" + f)

    jv_columns = {
        "Voltage (V)": df["Voltage (V)"],
        "Current Density (mA/cm^2)": df["Forward_mean (mA)"] / AREA
    }

    #Name each data set without the .csv
    name = f.split("/)[-1].replace(".csv", "")
    jv_data[name] = pd.DataFrame(jv_columns)

#Create a plot for the data

plt.figure(figsize=(7,5))

#Plot all of the datasets (loop), not just one

for name, jv in jv_data.items():
    plt.plot(jv["Voltage (V)"], jv["Current Density (mA/cm^2)"], ".",
↳label=name)

```

```

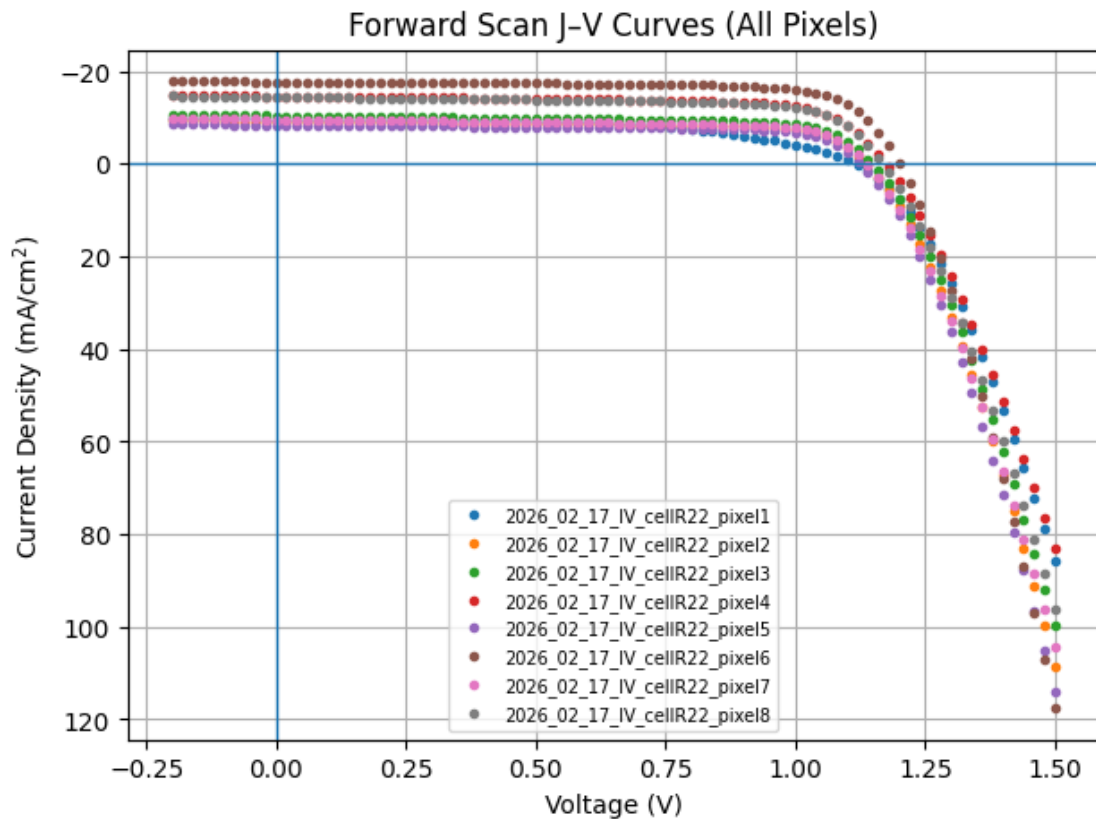
#Plot attributes

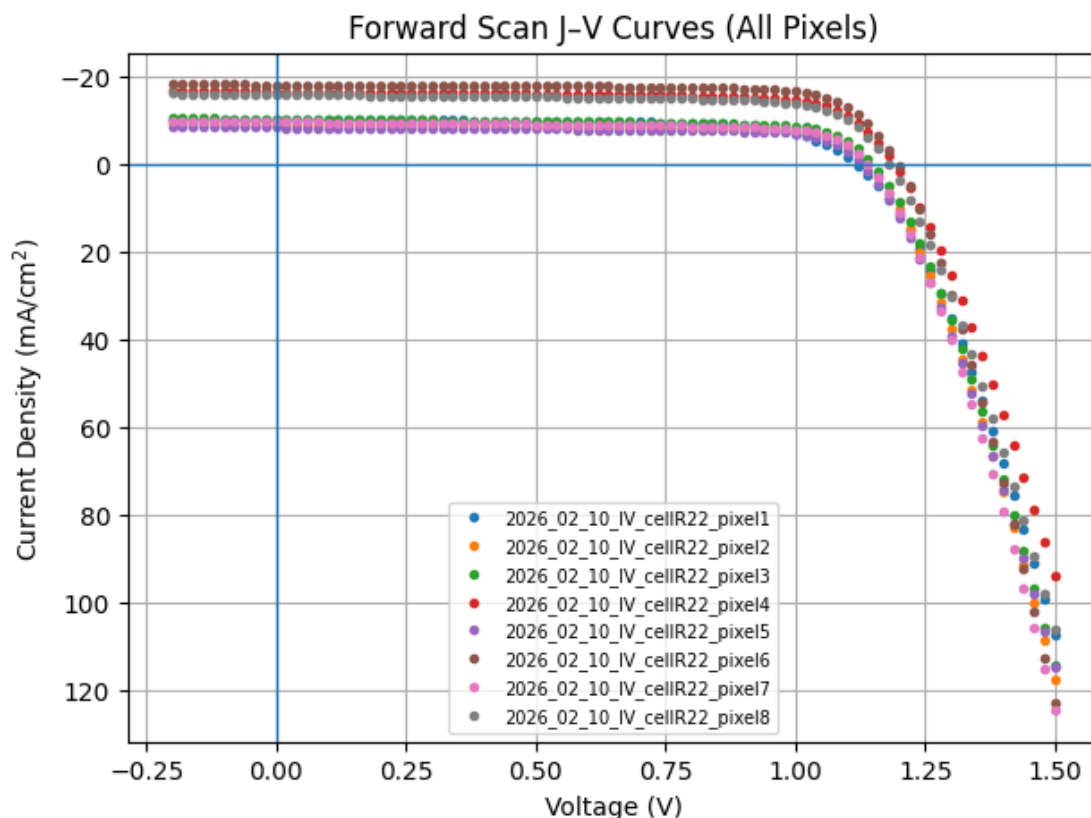
plt.axhline(0, linewidth=1)
plt.axvline(0, linewidth=1)
plt.xlabel("Voltage (V)")
plt.ylabel("Current Density (mA/cm$^2$)")
plt.title("Forward Scan J-V Curves (All Pixels)")
plt.legend(fontsize=7)
plt.grid(True)

# I inverted the axis bc that's how they showed this data in lecture
plt.gca().invert_yaxis()
plt.show()

```

Mounted at /content/drive





2. Calculate the J_{sc} and PCE values for each pixel to a CSV file. The code below will also extract the standard error (SE) of the current at key measurement points, which we will use for uncertainty propagation later in the notebook. The code will loop through this week's JV measurements and output a CSV file with the following column headers:

Pixel Number | J_{sc} (mA/cm²) | SE_ J_{sc} (mA/cm²) | J_{pmax} (mA/cm²) | V_{pmax} (V) | PCE | SE_ J_{pmax} (mA/cm²) | dV (V) | n_at_MPP

Note. If you already have your own loop to do these calculations, feel free to use that. Just make sure your code also reads the `Forward_std` (mA) and `Forward_n` columns from the raw data so you can compute the standard error.

7 J_{sc} and PCE for each pixel saved as .csv

```
[ ]: # Variables needed for J-V analysis
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Update variables as needed
date_str = "2026_02_17"
```

```

cell_id = "R22"
area_cm2 = 0.14
P_in = 99.8
pixels = range(1, 9)

# Column names matching the CSV output from measurement software
# JV data columns: Voltage (V), Forward_mean (mA), Forward_std (mA), Forward_n,
#                   Reverse_mean (mA), Reverse_std (mA), Reverse_n
COL_V = "Voltage (V)"
COL_Forward_mA = "Forward_mean (mA)"
COL_Fwd_std = "Forward_std (mA)"
COL_Fwd_n = "Forward_n"

# Store raw DataFrames and JV DataFrames for each pixel
raw_data = {}
jv_data = {}

# Load each pixel's data
for n in pixels:
    fname = f"{date_str}_IV_cell{cell_id}_pixel{n}.csv"
    try:
        df_raw = pd.read_csv(fname)
        raw_data[n] = df_raw # keep the full raw data for SE calculations

        df_jv = pd.DataFrame({
            "Voltage (V)": df_raw[COL_V].astype(float),
            "Current Density (mA/cm^2)": df_raw[COL_Forward_mA].astype(float) /
↪ area_cm2
        }).sort_values("Voltage (V)").reset_index(drop=True)

        # Add power density
        df_jv["Power Density (mW/cm^2)"] = (
            df_jv["Voltage (V)"] * df_jv["Current Density (mA/cm^2)"]
        )

        jv_data[n] = df_jv
        print(f"[ok] Loaded pixel {n}")

    except FileNotFoundError:
        print(f"[skip] Pixel {n}: file not found ({fname})")
    except Exception as e:
        print(f"[skip] Pixel {n}: error reading {fname}: {e}")

# --- Calculate PCE parameters and SE-based uncertainties ---
rows = []
valid_pixels = sorted(jv_data.keys())

```

```

for n in valid_pixels:
    jv = jv_data[n]
    raw = raw_data[n].sort_values(COL_V).reset_index(drop=True)

    V = jv["Voltage (V)"].to_numpy()
    J = jv["Current Density (mA/cm^2)"].to_numpy()

    # --- Standard PV parameters ---
    Voc = float(np.interp(0.0, J, V))
    Jsc = float(np.interp(0.0, V, J))
    P = V * J
    idx_pmax = int(np.argmax(P)) # most negative power = MPP
    Vmp = float(V[idx_pmax])
    Jmp = float(J[idx_pmax])

    denom = abs(Jsc) * Voc
    FF = (Vmp * abs(Jmp)) / denom if denom != 0 else np.nan
    PCE = ((Voc * abs(Jsc) * FF) / P_in) * 100 if P_in != 0 else np.nan

    # --- Standard Error at the MPP ---
    std_mA_mpp = float(raw[COL_Fwd_std].iloc[idx_pmax])
    n_mpp = float(raw[COL_Fwd_n].iloc[idx_pmax])

    if n_mpp > 1:
        SE_I_mpp = std_mA_mpp / np.sqrt(n_mpp) # SE of current in mA
        SE_J_mpp = SE_I_mpp / area_cm2 # SE of current density in
        ↪ mA/cm^2
    else:
        SE_J_mpp = np.nan
        print(f" [warn] Pixel {n}: n=1 at MPP, SE undefined")

    # --- Standard Error at V=0 (for Jsc uncertainty) ---
    idx_v0 = int(np.argmax(np.abs(raw[COL_V].to_numpy())))
    std_mA_v0 = float(raw[COL_Fwd_std].iloc[idx_v0])
    n_v0 = float(raw[COL_Fwd_n].iloc[idx_v0])

    if n_v0 > 1:
        SE_Jsc = (std_mA_v0 / np.sqrt(n_v0)) / area_cm2
    else:
        SE_Jsc = np.nan

    # --- Voltage uncertainty: half the voltage step size ---
    if len(V) > 1:
        voltage_step = float(np.median(np.abs(np.diff(V))))
        dV = voltage_step / 2.0
    else:
        dV = np.nan

```

```

rows.append({
    "Pixel Number":      n,
    "Jsc (mA/cm^2)":     Jsc,
    "SE_Jsc (mA/cm^2)":  SE_Jsc,
    "J_pmax (mA/cm^2)":  Jmp,
    "V_pmax (V)":        Vmp,
    "PCE":               PCE,
    "SE_Jpmax (mA/cm^2)": SE_J_mpp,
    "dV (V)":            dV,
    "n_at_MPP":          n_mpp,
})

# Convert to DataFrame and write out as CSV
out_name = f"{date_str}_jsc_pce_cell{cell_id}.csv"
df_out = pd.DataFrame(rows)

if df_out.empty:
    print("[warn] No valid pixels found - did you run the loader above?")
else:
    df_out.sort_values("Pixel Number").to_csv(out_name, index=False)
    print(f"\n[ok] Wrote {out_name}")
    display(df_out)

```

```

[ok] Loaded pixel 1
[ok] Loaded pixel 2
[ok] Loaded pixel 3
[ok] Loaded pixel 4
[ok] Loaded pixel 5
[ok] Loaded pixel 6
[ok] Loaded pixel 7
[ok] Loaded pixel 8

```

```
[ok] Wrote 2026_02_17_jsc_pce_cellR22.csv
```

	Pixel Number	Jsc (mA/cm ²)	SE_Jsc (mA/cm ²)	J_pmax (mA/cm ²)	\
0	1	-9.594929	0.002828	-8.025000	
1	2	-9.111214	0.003395	-7.677071	
2	3	-10.375214	0.005166	-8.779286	
3	4	-14.612286	0.004752	-12.742857	
4	5	-8.382929	0.003722	-7.166857	
5	6	-17.707286	0.005037	-15.745429	
6	7	-9.575500	0.003393	-7.943571	
7	8	-14.401643	0.003937	-12.510214	

	V_pmax (V)	PCE	SE_Jpmax (mA/cm ²)	dV (V)	n_at_MPP
0	0.76	6.111222	0.002600	0.01	10.0
1	0.98	7.538607	0.006022	0.01	10.0

2	0.98	8.620942	0.005265	0.01	10.0
3	0.98	12.513026	0.005369	0.01	10.0
4	0.96	6.893971	0.004079	0.01	10.0
5	1.02	16.092522	0.004574	0.01	10.0
6	0.98	7.800301	0.003162	0.01	10.0
7	0.98	12.284579	0.005690	0.01	10.0

NOTE. Even though your code gives you confirmation that it has successfully written your CSV, always open the file and check to make sure the data you meant to write is actually there.

Measurement Uncertainty and Error Bars

Last week in lecture, we discussed error propagation and the importance of measurement uncertainty in the context of asking and answering scientific questions. To this end, we will calculate uncertainties in our JV measurements, propagate those uncertainties through our analysis, and then include error bars in our plots.

Standard Error of the Mean When we take repeated measurements of the same quantity, we expect some variability from one measurement to the next. Two important statistical quantities help us characterize this variability:

Standard Deviation (std) tells us how spread out individual measurements are. If you measured a current 10 times, the standard deviation describes the typical spread of those 10 values.

Standard Error of the Mean (SE) tells us how precisely we know the *average* of those measurements:

$$SE = \frac{\text{std}}{\sqrt{n}}$$

where n is the number of repeated measurements. As we take more measurements, the standard error gets smaller — our estimate of the mean becomes more precise.

Why SE instead of std? When we report J_{pmax} , J_{sc} , or any other parameter, we are using the *mean* value from our repeated measurements. The appropriate uncertainty for a mean value is the standard error, not the standard deviation.

Our measurement software already computes the mean, standard deviation, and n for each voltage point. The columns `Forward_std` (mA) and `Forward_n` in your J-V data files contain exactly what we need:

$$SE_J = \frac{\text{Forward_std (mA)}}{\sqrt{\text{Forward_n}}} \div \text{pixel area}$$

The code above already extracted these SE values at the maximum power point and at $V = 0$.

Uncertainty Propagation for PCE We will propagate the uncertainty using the following formula:

$$\delta f = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 (\delta x)^2 + \left(\frac{\partial f}{\partial y}\right)^2 (\delta y)^2 + \left(\frac{\partial f}{\partial z}\right)^2 (\delta z)^2}$$

Recall that:

$$PCE = \frac{|J_{pmax}| \times V_{pmax}}{P_{in}} \times 100\%$$

so the uncertainty in the PCE is:

$$\delta_{PCE} = PCE \times \sqrt{\left(\frac{\delta J_{pmax}}{J_{pmax}}\right)^2 + \left(\frac{\delta V_{pmax}}{V_{pmax}}\right)^2 + \left(\frac{\delta P_{in}}{P_{in}}\right)^2}$$

where: - δJ_{pmax} = standard error of the current density at the maximum power point (from our data) - δV_{pmax} = half the voltage step size (instrument resolution) - $\delta P_{in} = 0.9 mW/cm^2$ (calibration uncertainty of the solar simulator)

The code in the next cell computes this propagation automatically using the SE values extracted above.

Which uncertainty matters most? Notice that we have three very different *types* of uncertainty entering the propagation:

Source	Type	What it represents
δJ_{pmax}	Statistical (SE from repeated measurements)	How precisely we know the mean current at the MPP
δV_{pmax}	Instrumental (voltage step resolution)	The finite spacing between voltage set-points
δP_{in}	Calibration (solar simulator spec)	How well we know the incident light power

After running the code below, look at the relative uncertainty breakdown it prints. Which of these three terms dominates the total PCE uncertainty? What does that tell you about what limits the precision of your measurement — and what you would need to change to reduce it?

8 Jsc and PCE uncertainty w/ error

```
[ ]: # --- PCE Uncertainty Propagation using Standard Error ---
df = pd.read_csv(f"{date_str}_jsc_pce_cell{cell_id}.csv")

# Time variable - update depending on the week
t = 168 # hours since initial measurement (update each week)
df["Time (hr)"] = t

# Uncertainty in light power (from solar simulator calibration)
dP_in = 0.9 # mW/cm^2

# --- Propagate uncertainty for PCE ---
J_abs = np.abs(df["J_pmax (mA/cm^2)"].to_numpy())
V_vals = np.abs(df["V_pmax (V)"].to_numpy())
PCE_vals = df["PCE"].to_numpy()
dJ = df["SE_Jpmax (mA/cm^2)"].to_numpy()
dV = df["dV (V)"].to_numpy()
```

```

# Relative uncertainties
rel_J    = dJ / J_abs
rel_V    = dV / V_vals
rel_Pin  = dP_in / P_in

# PCE uncertainty (in percentage points, same units as PCE)
df["PCE_uncertainty (%)"] = PCE_vals * np.sqrt(rel_J**2 + rel_V**2 + rel_Pin**2)

# Jsc uncertainty for reporting
df["Jsc_uncertainty (mA/cm^2)"] = df["SE_Jsc (mA/cm^2)"]

# Write final output
out_name = f"{date_str}_jsc_pce_uncertainties_cell{cell_id}.csv"
cols_to_write = [
    "Pixel Number", "Time (hr)",
    "Jsc (mA/cm^2)", "Jsc_uncertainty (mA/cm^2)",
    "PCE", "PCE_uncertainty (%)"
]
df[cols_to_write].to_csv(out_name, index=False)
print(f"[ok] Wrote {out_name}")
display(df[cols_to_write])

# Print a summary for quick review
print("\n--- Summary ---")
for _, row in df.iterrows():
    pix = int(row["Pixel Number"])
    pce = row["PCE"]
    dpce = row["PCE_uncertainty (%)"]
    jsc = row["Jsc (mA/cm^2)"]
    djsc = row["Jsc_uncertainty (mA/cm^2)"]
    print(f" Pixel {pix}: PCE = {pce:.2f} +/- {dpce:.2f} %,   "
          f"Jsc = {jsc:.3f} +/- {djsc:.3f} mA/cm^2")

# --- Relative uncertainty breakdown ---
print("\n--- Relative Uncertainty Breakdown (as %) ---")
print(f" {'Pixel':>5s} {' J/J':>8s} {' V/V':>8s} {' P/P':>8s} {'Total':>8s}")
for _, row in df.iterrows():
    pix = int(row["Pixel Number"])
    rJ = row["SE_Jpmax (mA/cm^2)"] / abs(row["J_pmax (mA/cm^2)"]) * 100
    rV = row["dV (V)"] / abs(row["V_pmax (V)"]) * 100
    rP = dP_in / P_in * 100
    total = np.sqrt((rJ/100)**2 + (rV/100)**2 + (rP/100)**2) * 100
    print(f" {pix:5d} {rJ:7.3f}% {rV:7.3f}% {rP:7.3f}% {total:7.3f}%")

print(f"\n Note: J/J is the SE-based measurement uncertainty (statistical).")

```

```
print(f"          V/V is the voltage step resolution (instrumental).")
print(f"          P/P is the solar simulator calibration (systematic).")
```

[ok] Wrote 2026_02_17_jsc_pce_uncertainties_cellR22.csv

	Pixel Number	Time (hr)	Jsc (mA/cm ²)	Jsc_uncertainty (mA/cm ²) \
0	1	168	-9.594929	0.002828
1	2	168	-9.111214	0.003395
2	3	168	-10.375214	0.005166
3	4	168	-14.612286	0.004752
4	5	168	-8.382929	0.003722
5	6	168	-17.707286	0.005037
6	7	168	-9.575500	0.003393
7	8	168	-14.401643	0.003937

	PCE	PCE_uncertainty (%)
0	6.111222	0.097504
1	7.538607	0.102830
2	8.620942	0.117513
3	12.513026	0.170483
4	6.893971	0.095066
5	16.092522	0.214415
6	7.800301	0.106269
7	12.284579	0.167384

--- Summary ---

Pixel 1: PCE = 6.11 +/- 0.10 %, Jsc = -9.595 +/- 0.003 mA/cm²
Pixel 2: PCE = 7.54 +/- 0.10 %, Jsc = -9.111 +/- 0.003 mA/cm²
Pixel 3: PCE = 8.62 +/- 0.12 %, Jsc = -10.375 +/- 0.005 mA/cm²
Pixel 4: PCE = 12.51 +/- 0.17 %, Jsc = -14.612 +/- 0.005 mA/cm²
Pixel 5: PCE = 6.89 +/- 0.10 %, Jsc = -8.383 +/- 0.004 mA/cm²
Pixel 6: PCE = 16.09 +/- 0.21 %, Jsc = -17.707 +/- 0.005 mA/cm²
Pixel 7: PCE = 7.80 +/- 0.11 %, Jsc = -9.575 +/- 0.003 mA/cm²
Pixel 8: PCE = 12.28 +/- 0.17 %, Jsc = -14.402 +/- 0.004 mA/cm²

--- Relative Uncertainty Breakdown (as %) ---

Pixel	J/J	V/V	P/P	Total
1	0.032%	1.316%	0.902%	1.595%
2	0.078%	1.020%	0.902%	1.364%
3	0.060%	1.020%	0.902%	1.363%
4	0.042%	1.020%	0.902%	1.362%
5	0.057%	1.042%	0.902%	1.379%
6	0.029%	0.980%	0.902%	1.332%
7	0.040%	1.020%	0.902%	1.362%
8	0.045%	1.020%	0.902%	1.363%

Note: J/J is the SE-based measurement uncertainty (statistical).

V/V is the voltage step resolution (instrumental).
P/P is the solar simulator calibration (systematic).

Interpreting the Uncertainty Breakdown Look at the relative uncertainty table printed above. You should notice that:

1. $\delta J/J$ is **very small** (typically $< 0.1\%$). With n repeated measurements at each voltage, the standard error of the mean current is tiny — our instrument measures current very precisely and reproducibly.
2. $\delta V/V$ and $\delta P_{in}/P_{in}$ **dominate** (each $\sim 1\%$). These are *not* statistical uncertainties from repeated measurements — they are fixed properties of the instrument (the voltage step size chosen for the sweep) and the calibration of the solar simulator.

This reveals something important about this experiment: **the precision of the PCE measurement is limited by instrument resolution and calibration, not by measurement repeatability.** Taking more current measurements (increasing n) would make $\delta J/J$ even smaller, but would barely change the total uncertainty because the other two terms dominate.

Question 1.5: Looking at the breakdown, which term contributes the most to the total PCE uncertainty? If you wanted to cut the PCE uncertainty in half, what would you need to change about the measurement — and what would *not* help?

3. Repeat steps 1-2 for your other weeks of data to create similar CSV files (update `date_str` and `t` for each week).

Make **copies** of your JV measurement CSV files from the past several weeks to your Colab environment. **Do not remove the raw files from their original folder!**

Once you have all of your uncertainty CSV files, you are ready to plot.

9 Plotting Jsc and PCE uncertainties

```
[ ]: # Add CSV files you want to have on the plot
files = [
    "2026_01_27_jsc_pce_uncertainties_cellR22.csv",
    "2026_02_10_jsc_pce_uncertainties_cellR22.csv",
    "2026_02_17_jsc_pce_uncertainties_cellR22.csv",
    # etc,
]

# Load and combine data, with the check to identify and skip any missng files ↵
↳missing files
dfs = []
for f in files:
    try:
        d = pd.read_csv(f)
        d["source_file"] = f # optional: helps debug
        dfs.append(d)
    except FileNotFoundError:
```

```

        print(f"[skip] missing file: {f}")

if not dfs:
    raise SystemExit("No data files loaded. Check file names above.")

all_df = pd.concat(dfs, ignore_index=True)

# Keep only the columns we need and make them be numbers. You can add
↳ additional details from the input files if you want.
need = ["Pixel Number", "Time (hr)", "PCE", "PCE_uncertainty (%)"]
missing = [c for c in need if c not in all_df.columns]
if missing:
    raise ValueError(f"Missing required columns in data: {missing}")

all_df["Pixel Number"] = pd.to_numeric(all_df["Pixel Number"], errors="coerce").
    ↳ astype("Int64")
all_df["Time (hr)"] = pd.to_numeric(all_df["Time (hr)"], errors="coerce")
all_df["PCE"] = pd.to_numeric(all_df["PCE"], errors="coerce")
all_df["PCE_uncertainty (%)"] = pd.to_numeric(all_df["PCE_uncertainty (%)"],
    ↳ errors="coerce")

# Drop rows missing basics and dedupe (keep last if duplicates exist)
all_df = (
    all_df.dropna(subset=["Pixel Number", "Time (hr)", "PCE"])
    .drop_duplicates(subset=["Pixel Number", "Time (hr)"], keep="last")
)

# Determine which pixels actually appear
pixels_present = sorted(all_df["Pixel Number"].dropna().unique().astype(int))

# Make a plot
fig, ax = plt.subplots(figsize=(9, 6))

markers = ["o", "s", "^", "D", "v", ">", "x", "*"]

for i, pix in enumerate(pixels_present):
    d = all_df[all_df["Pixel Number"] == pix].sort_values("Time (hr)")
    if d.empty:
        continue

    marker = markers[i % len(markers)] # cycle markers if many pixels

    ax.errorbar(
        d["Time (hr)"].to_numpy(),
        d["PCE"].to_numpy(),
        yerr=d["PCE_uncertainty (%)"].to_numpy(),
        fmt=marker,

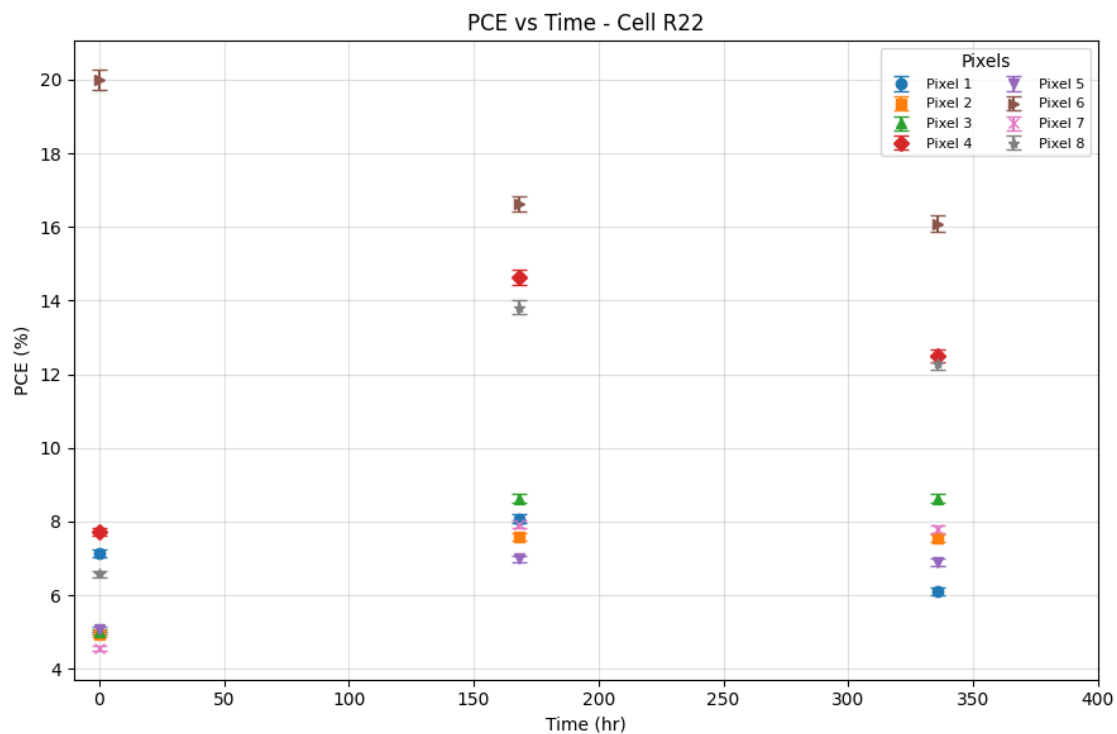
```

```

        capsize=4,
        label=f"Pixel {pix}"
    )
# Update legend/labels as desired
ax.set_xlim(-10, 400)
ax.set_xlabel("Time (hr)")
ax.set_ylabel("PCE (%)")
ax.set_title(f"PCE vs Time - Cell {cell_id}")
ax.grid(True, alpha=0.4)
ax.legend(fontsize=8, title="Pixels", ncols=2)

plt.tight_layout()
plt.show()

```



Congratulations! That was a bunch of new code, but you will have to make minimal edits to use it in upcoming weeks.

Question 2: Discuss what you observe from looking at the PCE over time. Has anything changed over the past few weeks?

The PCE has gone up substantially from week one. The difference between week 3 and 4 has been much more subtle, where we have even seen some of the pixels decreasing in PCE. Our experiments and data collection hasn't changed any so I assume it has to do with degradation of the pixel or a possible flux in the temperature.

9.1 EQE Analysis with Uncertainty

The External Quantum Efficiency at each wavelength is:

$$EQE(\lambda) = \frac{I(\lambda)}{P(\lambda)} \times \frac{hc}{e\lambda}$$

where $I(\lambda)$ is the measured photocurrent and $P(\lambda)$ is the incident optical power at wavelength λ .

Since EQE is a ratio of two measured quantities (each with their own uncertainty), we can propagate the uncertainties using the same approach as for PCE:

$$\frac{\delta EQE}{EQE} = \sqrt{\left(\frac{\delta I}{I}\right)^2 + \left(\frac{\delta P}{P}\right)^2}$$

$$\text{where: } \delta I = SE_I = \frac{\text{Current_std (nA)}}{\sqrt{n_I}} - \delta P = SE_P = \frac{\text{Power_std (uW)}}{\sqrt{n_P}}$$

This gives us an uncertainty band around each EQE curve, allowing us to assess whether changes in EQE between weeks are statistically significant.

As a reminder, the EQE data files have the following columns: - Power data: Wavelength (nm), Power_mean (uW), Power_std (uW), n - Current data: Wavelength (nm), Current_mean (nA), Current_std (nA), n

- 1) The code below calculates EQE and its uncertainty for each pixel and saves the results to a CSV file.
- 2) The plotting cells that follow create EQE curves with shaded uncertainty bands.

10 EQE and uncertainty

```
[ ]: # --- EQE Calculation with Standard Error Uncertainty ---

# Physical constants
h = 6.62607015e-34      # Planck's constant (J*s)
c = 3.0e8               # Speed of light (m/s)
e_charge = 1.602176634e-19 # Elementary charge (C)

# Load power data (shared across all pixels for one apparatus)
power_fname = f"{date_str}_power_cell{cell_id}.csv"
try:
    power_data = pd.read_csv(power_fname)
    print(f"[ok] Loaded power data: {power_fname}")
except FileNotFoundError:
    print(f"[error] Power file not found: {power_fname}")
    print("  Update the filename above to match your power data file.")

wavelength_nm = power_data["Wavelength (nm)"].to_numpy()
wavelength_m = wavelength_nm * 1e-9

P_mean_uW = power_data["Power_mean (uW)"].to_numpy()
P_std_uW = power_data["Power_std (uW)"].to_numpy()
n_P = power_data["n"].to_numpy().astype(float)
```



```

# SE of power
SE_P_uW = np.where(n_P > 1, P_std_uW / np.sqrt(n_P), np.nan)

# Build EQE results dataframe
eqe_df = pd.DataFrame({"Wavelength (nm)": wavelength_nm})

for pix in sorted(jv_data.keys()):
    curr_fname = f"{date_str}_current_cell{cell_id}_pixel{pix}.csv"
    try:
        curr_data = pd.read_csv(curr_fname)

        I_mean_nA = curr_data["Current_mean (nA)"].to_numpy()
        I_std_nA = curr_data["Current_std (nA)"].to_numpy()
        n_I = curr_data["n"].to_numpy().astype(float)

        # SE of current
        SE_I_nA = np.where(n_I > 1, I_std_nA / np.sqrt(n_I), np.nan)

        # Convert to SI units
        I_mean_A = I_mean_nA * 1e-9
        SE_I_A = SE_I_nA * 1e-9
        P_mean_W = P_mean_uW * 1e-6
        SE_P_W = SE_P_uW * 1e-6

        # EQE calculation
        eqe_vals = (I_mean_A / P_mean_W) * (h * c) / (e_charge * wavelength_m)

        # EQE uncertainty propagation: dEQE/EQE = sqrt((dI/I)^2 + (dP/P)^2)
        rel_I = SE_I_A / np.abs(I_mean_A)
        rel_P = SE_P_W / np.abs(P_mean_W)
        eqe_unc = np.abs(eqe_vals) * np.sqrt(rel_I**2 + rel_P**2)

        eqe_df[f"eqe_pix{pix}"] = eqe_vals
        eqe_df[f"eqe_unc_pix{pix}"] = eqe_unc

        print(f"[ok] Pixel {pix}: peak EQE = {np.nanmax(eqe_vals):.4f} "
              f"({np.nanmax(eqe_vals)*100:.1f}%)")
    except FileNotFoundError:
        print(f"[skip] Pixel {pix}: current file not found ({curr_fname})")
    except Exception as e:
        print(f"[skip] Pixel {pix}: {e}")

# Write to CSV
eqe_out = f"{date_str}_eqe_cell{cell_id}.csv"
eqe_df.to_csv(eqe_out, index=False)
print(f"\n[ok] Wrote {eqe_out}")

```

```
[ok] Loaded power data: 2026_02_17_power_cellR22.csv
[ok] Pixel 1: peak EQE = 0.8380 (83.8%)
[ok] Pixel 2: peak EQE = 0.8692 (86.9%)
[ok] Pixel 3: peak EQE = 0.9003 (90.0%)
[ok] Pixel 4: peak EQE = 0.9384 (93.8%)
[ok] Pixel 5: peak EQE = 0.8631 (86.3%)
[ok] Pixel 6: peak EQE = 1.0203 (102.0%)
[ok] Pixel 7: peak EQE = 0.8665 (86.6%)
[ok] Pixel 8: peak EQE = 0.9345 (93.4%)
```

```
[ok] Wrote 2026_02_17_eqe_cellR22.csv
```

```
[ ]: # --- EQE Plots with Shaded Uncertainty Bands (This Week) ---
eqe_df_plot = pd.read_csv(f"{date_str}_eqe_cell{cell_id}.csv")
wl = eqe_df_plot["Wavelength (nm)"].to_numpy()

fig, axes = plt.subplots(4, 2, figsize=(12, 16))
axes_flat = axes.flatten()

for i in range(8):
    ax = axes_flat[i]
    pix = i + 1
    col_eqe = f"eqe_pix{pix}"
    col_unc = f"eqe_unc_pix{pix}"

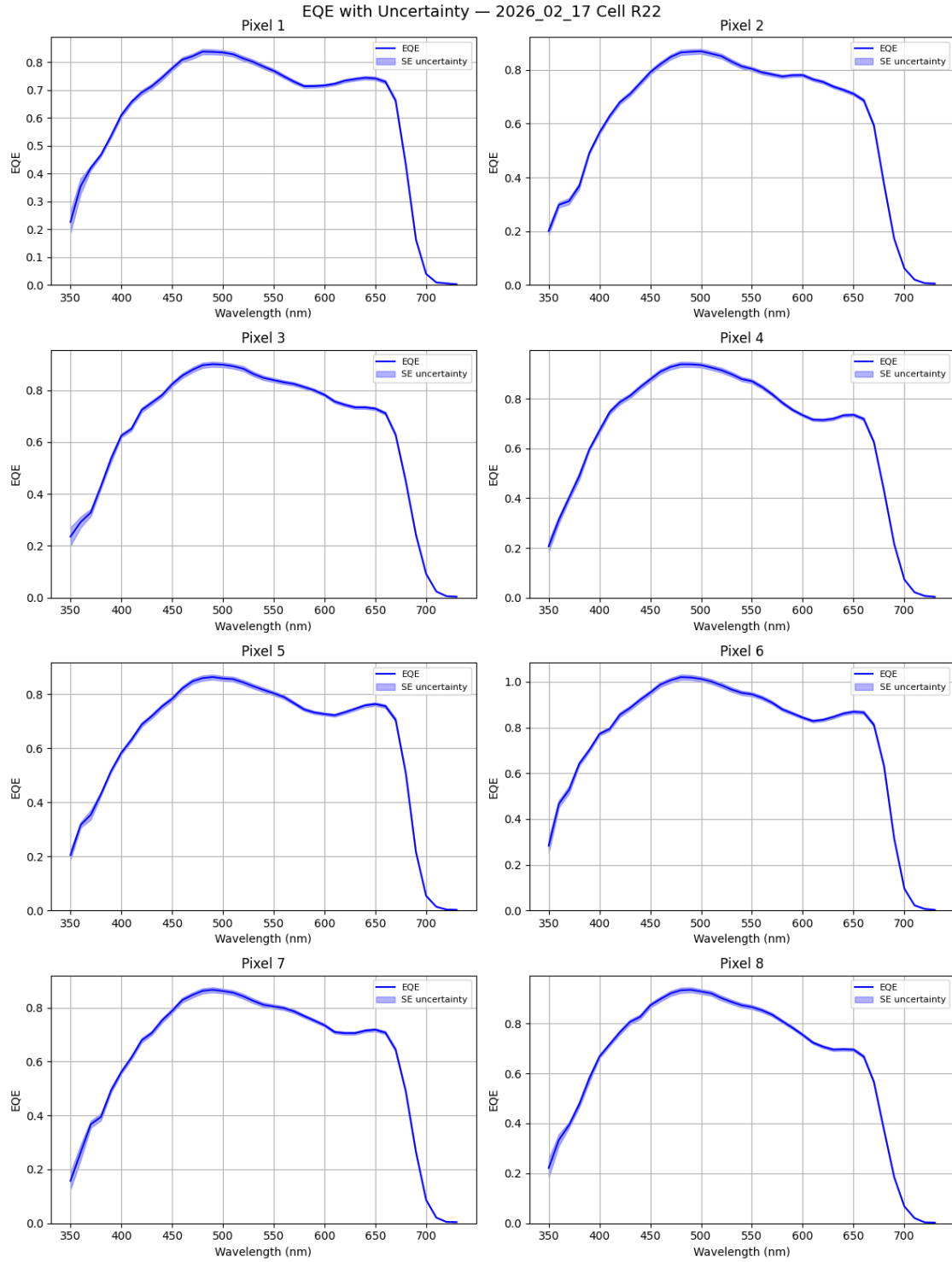
    if col_eqe not in eqe_df_plot.columns:
        ax.set_visible(False)
        continue

    eqe_vals = eqe_df_plot[col_eqe].to_numpy()
    ax.plot(wl, eqe_vals, 'b-', label="EQE")

    if col_unc in eqe_df_plot.columns:
        eqe_unc = eqe_df_plot[col_unc].to_numpy()
        ax.fill_between(wl,
                        eqe_vals - eqe_unc,
                        eqe_vals + eqe_unc,
                        alpha=0.3, color='blue',
                        label="SE uncertainty")

    ax.set_title(f"Pixel {pix}")
    ax.set_xlabel("Wavelength (nm)")
    ax.set_ylabel("EQE")
    ax.legend(fontsize=8)
    ax.set_ylim(bottom=0)
    ax.grid(True)
```

```
plt.suptitle(f"EQE with Uncertainty - {date_str} Cell {cell_id}", fontsize=14)
plt.tight_layout()
plt.show()
```



```
[ ]: # --- Multi-Week EQE Comparison with Uncertainty Bands ---

# Add your EQE CSV files here (update filenames each week)
eqe_files = {
    "Week 2": "/content/2026_02_03_eqe_cellR22.csv",
    "Week 3": "/content/2026_02_10_eqe_cellR22.csv",
    "Week 4": "/content/2026_02_17_eqe_cellR22.csv",
    # Add more weeks as needed
}

# Color cycle for different weeks
colors = ['blue', 'red', 'green', 'purple', 'orange', 'brown']

# Load all EQE files
eqe_weeks = {}
for label, fname in eqe_files.items():
    try:
        eqe_weeks[label] = pd.read_csv(fname)
        print(f"[ok] Loaded {label}: {fname}")
    except FileNotFoundError:
        print(f"[skip] {label}: {fname} not found")

if not eqe_weeks:
    print("[warn] No EQE files loaded. Check filenames above.")
else:
    fig, axes = plt.subplots(4, 2, figsize=(12, 16))
    axes_flat = axes.flatten()

    for i in range(8):
        ax = axes_flat[i]
        pix = i + 1
        col_eqe = f"eqe_pix{pix}"
        col_unc = f"eqe_unc_pix{pix}"

        for j, (label, df_eqe) in enumerate(eqe_weeks.items()):
            color = colors[j % len(colors)]
            wl = df_eqe["Wavelength (nm)"].to_numpy()

            if col_eqe in df_eqe.columns:
                eqe_vals = df_eqe[col_eqe].to_numpy()
                ax.plot(wl, eqe_vals, color=color, label=label)

            # Add uncertainty band if the column exists
            if col_unc in df_eqe.columns:
                eqe_unc = df_eqe[col_unc].to_numpy()
                ax.fill_between(wl,
                               eqe_vals - eqe_unc,
```

```

                                eqe_vals + eqe_unc,
                                alpha=0.15, color=color)

    ax.set_title(f"Pixel {pix}")
    ax.set_xlabel("Wavelength (nm)")
    ax.set_ylabel("EQE")
    ax.legend(fontsize=7)
    ax.set_ylim(bottom=0)
    ax.grid(True)

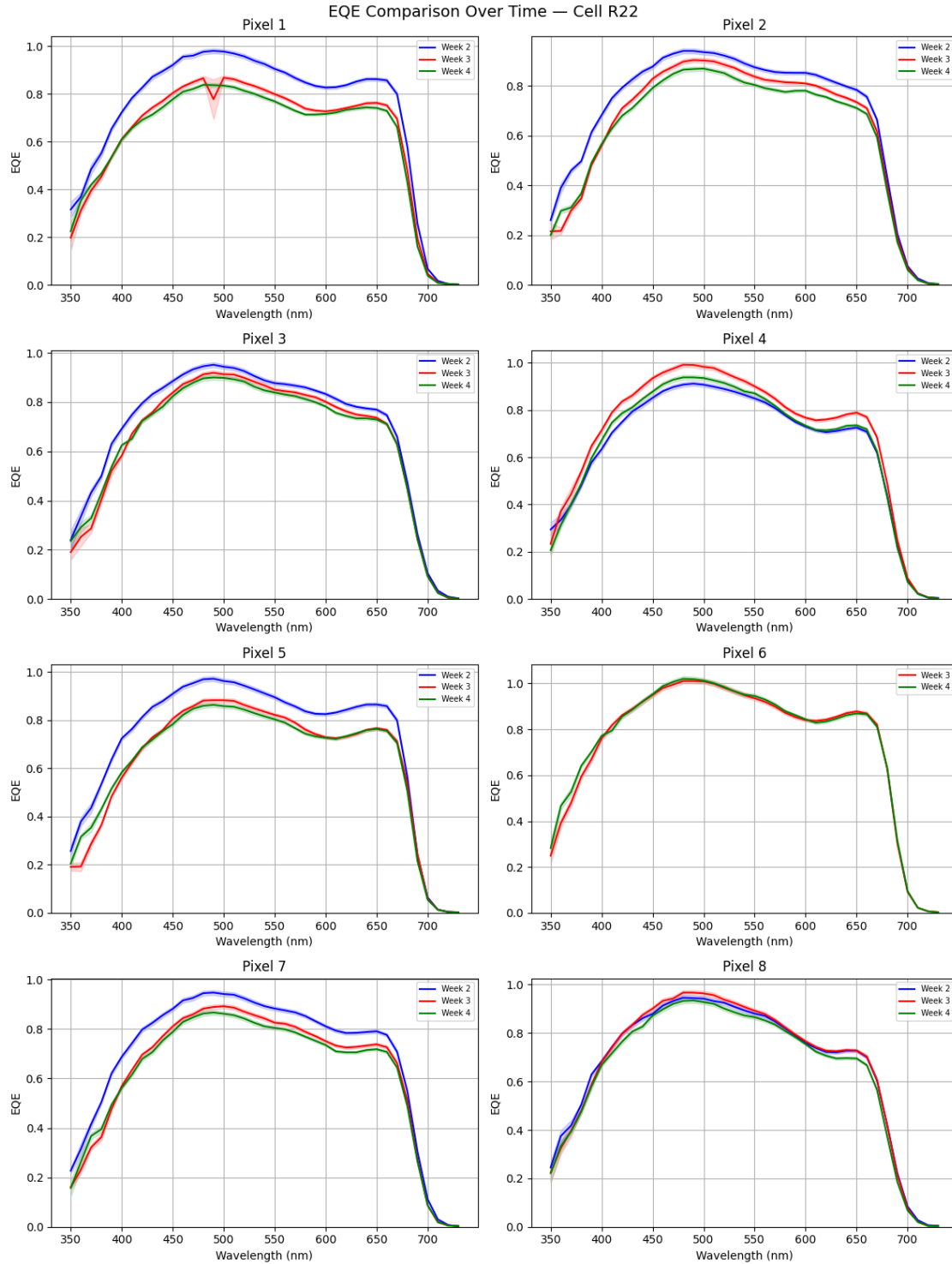
plt.suptitle(f"EQE Comparison Over Time - Cell {cell_id}", fontsize=14)
plt.tight_layout()
plt.show()

```

```

[ok] Loaded Week 2: /content/2026_02_03_eqe_cellR22.csv
[ok] Loaded Week 3: /content/2026_02_10_eqe_cellR22.csv
[ok] Loaded Week 4: /content/2026_02_17_eqe_cellR22.csv

```



Question 3: Discuss what you observe from looking at the comparison of all of your EQE curves. Considering the uncertainty bands, are the changes in EQE between weeks statistically significant? At which wavelengths is the uncertainty largest relative to the EQE value, and why?

The change in EQE has actually not been drastic in any way. It seems that our week 2 pixels have had the largest change in EQE, but only in some of the pixels. Overall the EQE is pretty similar from week to week. The only significant change that I am noticing our pixel 1 had a rapid drop in EQE around the 475nm range but then promptly shoots back up and is back to normal just after the 500nm range.

The uncertainty is the targets around the 500nm range for all of the pixels, I believe this is likely because that is around where the max EQE is for our cell.

11 Author Contributions (required)

1. List the name of each team member and please describe briefly how each member contributed to the work in lab this week. (e.g., taking a measurement, updating the Colab notebook, etc.)

Greg - Completed majority of coding, assisted in data collection. Jay - Adjusted code of previous weeks to get proper csv files, assisted in data collection. Madelyn - Assisted in interpreting data and answering questions.

12 Use of AI (required)

1. How your team used AI tools: Describe the specific tasks where AI-assisted tools were involved. For example, did AI help with writing, debugging, optimizing, or refining your code? Which components of the analysis did your team use AI on?

This week there was no need for any AI usage as it was mostly copying and modifying previous code.

2. When your team used AI tools: Specify at what stage(s) of your programming process you used AI. Was it during initial code development, troubleshooting, etc.?

This week our team used zero AI tools

3. Which AI tools your team used: Identify the AI tools or platforms your team consulted (e.g., ChatGPT, GitHub Copilot, etc.).

None